

Below is a concise, complete, MSDN-style document describing the data model for a financial planning system based on the provided query and thinking trace.

Financial Planning Data Model

Overview

This document defines the data model for a personal financial planning system. It includes entities, their fields, relationships, and computed values to support planning, tracking, forecasting, and scenario analysis. The model ensures flexibility, usability, and support for all required views.

Entities

1. Plans

Purpose: Represents financial plans (e.g., budgets, savings, funds).

Fields:

- **id:** Unique identifier (int, primary key).
- **name:** Plan name (string, required).
- **type:** Plan type (enum: **income**, **expense**, **saving**, required).
- **amount:** Target or expected amount (decimal, required).
- **frequency:** Recurrence (enum: **one-time**, **daily**, **weekly**, **monthly**, **yearly**, required).
- **start_date:** Start date (date, required).
- **end_date:** End date (date, optional).
- **current_balance:** Current balance (decimal, computed).
- **parent_id:** Parent plan reference (int, foreign key to **plans.id**, optional).
- **goal_id:** Linked goal (int, foreign key to **goals.id**, optional).

Relationships:

- One-to-many with **transactions**.
- Many-to-many with **wallets** (via junction table).
- One-to-many with **forecasts**.
- One-to-many with **goals**.

2. Transactions

Purpose: Tracks actual financial operations (income, expenses, transfers).

Fields:

- **id:** Unique identifier (int, primary key).
- **type:** Transaction type (enum: **income**, **expense**, **transfer**, required).
- **amount:** Amount (decimal, required).
- **date:** Transaction date (date, required).
- **category:** Category (string, optional).
- **description:** Description (string, optional).
- **plan_id:** Linked plan (int, foreign key to **plans.id**, optional).
- **wallet_id:** Source/destination wallet (int, foreign key to **wallets.id**, required).

- **is_virtual**: Virtual transfer flag (bool, default: **false**).

Relationships:

- Many-to-one with **plans**.
- Many-to-one with **wallets**.

3. Wallets

Purpose: Represents money storage (e.g., bank accounts, cash).

Fields:

- **id**: Unique identifier (int, primary key).
- **name**: Wallet name (string, required).
- **current_balance**: Current balance (decimal, computed).

Relationships:

- One-to-many with **transactions**.
- One-to-many with **inventories**.

4. Inventories

Purpose: Snapshots of wallet balances at specific dates.

Fields:

- **id**: Unique identifier (int, primary key).
- **wallet_id**: Linked wallet (int, foreign key to **wallets.id**, required).
- **date**: Snapshot date (date, required).
- **balance**: Balance at date (decimal, required).

Relationships:

- Many-to-one with **wallets**.

5. Goals

Purpose: Defines financial goals tied to plans.

Fields:

- **id**: Unique identifier (int, primary key).
- **plan_id**: Linked plan (int, foreign key to **plans.id**, required).
- **target_amount**: Target amount (decimal, required).
- **target_date**: Target date (date, required).
- **progress**: Progress percentage (decimal, computed).

Relationships:

- Many-to-one with **plans**.

6. Forecasts

Purpose: Predicted values for plans or overall state.

Fields:

- **id**: Unique identifier (int, primary key).
- **plan_id**: Linked plan (int, foreign key to **plans.id**, optional).
- **date**: Forecast date (date, required).

- **forecast_amount**: Predicted amount (decimal, required).

Relationships:

- Many-to-one with **plans**.

7. Scenarios

Purpose: Alternative data sets for "what-if" modeling.

Fields:

- **id**: Unique identifier (int, primary key).
- **name**: Scenario name (string, required).
- **user_id**: Owner (int, foreign key to **users.id**, required).
- **created_at**: Creation date (date, required).

Relationships:

- One-to-many with **plans** (copies for scenario).
- One-to-many with **forecasts** (copies for scenario).

Computed Values

1. Plan Balance on Date

Purpose: Shows plan's state at a specific date.

Calculation: Sum of incoming **transactions** (**type** = **income** or **transfer** into plan) minus sum of outgoing **transactions** (**type** = **expense** or **transfer** out) up to the date.

Rules: Includes both real and virtual transactions.

2. Forecasted Plan Balance

Purpose: Predicts future plan balance.

Calculation: Uses **forecasts.forecast_amount** for the plan or aggregates general forecasts if **plan_id** is null.

Rules: Overrides computed balance for future dates.

3. Goal Progress

Purpose: Tracks goal achievement.

Calculation: $(\text{plans.current_balance} / \text{goals.target_amount}) * 100$.

Rules: Capped at 100%, null if no linked goal.

4. Plan vs. Actual Variance

Purpose: Identifies deviations from plan.

Calculation: **plans.amount** minus sum of related **transactions.amount** for the period.

Rules: Considers only real transactions.

5. Hierarchical Variance

Purpose: Aggregates deviations across plan hierarchy.

Calculation: Sum of variances of all child plans (recursive).

Rules: Null if no children.

Virtual Transfers

Purpose: Adjusts plan balances for reconciliation or scenario modeling.

Fields: (Stored as `transactions` with `is_virtual = true`)

- **type:** Fixed as `"virtual transfer"`.
 - **amount:** Transfer amount (decimal, required).
 - **date:** Transfer date (date, required).
 - **source_plan_id:** Source plan (int, foreign key to `plans.id`, required).
 - **target_plan_id:** Target plan (int, foreign key to `plans.id`, required).
- Behavior:** Affects `current_balance` but not `wallets`.

Design Considerations

- **Evolution:** Entities support updates (e.g., `end_date`, `amount`) and hierarchies (`parent_id`).
- **Views:** Filters by date, type, and relationships enable all required views (e.g., planned expenses, actual distribution).
- **Usability:** Minimal fields, clear relationships, and computed values simplify management.

Open Questions

- **Scenario Storage:** Option 1: Full data copy; Option 2: Deltas from base (preferred for efficiency, undecided).
- **Performance:** Large transaction volumes may require cached balances (undecided).
- **History:** Versioning or logging changes possible but not implemented.

Unresolved Aspects

- UI details for view rendering (out of scope).
- Detailed performance optimization (to be addressed in implementation).